
Cross-Platform Transferability of Prompt Injection Attacks: A Taxonomy of Universal Vulnerabilities in Large Language Models

[Redacted for peer review]
Independent Security Research
[redacted]

Abstract

We present an empirical analysis of prompt injection attacks against large language models (LLMs), identifying ten adversarial prompts that achieve consistent success across 9–11 models spanning 4–6 inference platforms. Each attack exploits a distinct structural vulnerability in the instruction-following, persona-modeling, or context-management architecture of contemporary LLMs. We propose a six-category taxonomy of attack surfaces—behavioral control, jailbreak, persona hijack, cognitive state manipulation, encoding evasion, and semantic manipulation—and analyze the mechanistic reasons for cross-model transferability.

Our empirical battery—2,919 adversarial sessions across 18 model/harness combinations on 5 platforms—reveals three key findings. First, vulnerability is scale-dependent but not scale-eliminated: small models (4B parameters) show 90–100% attack effectiveness while frontier models show 8–10%. Second, the agent harness dominates the model: the same underlying model (Claude Haiku 4.5) shows 53% effectiveness with 0% canary exfiltration via Claude Code CLI versus 95% effectiveness with 50% canary exfiltration via the HERMES agent harness, demonstrating that harness-level guardrails provide safety guarantees that model-level alignment alone cannot achieve. Third, attacks leave detectable behavioral signatures: a lightweight classifier over runtime telemetry features (entropy, latency, throughput) achieves $AUC = 0.838$ without semantic analysis.

Our findings demonstrate that despite diverse training procedures and alignment strategies, current LLMs share fundamental architectural assumptions that create exploitable common-mode failures. We conclude with a defense-in-depth framework emphasizing harness-level instruction hierarchy, channel-aware input classification, and runtime telemetry anomaly detection.

1 Introduction

The deployment of large language models across consumer, enterprise, and critical infrastructure applications has created an urgent need for robust safety alignment. Despite significant investment in techniques such as Reinforcement Learning from Human Feedback (RLHF) [Ouyang et al., 2022], Constitutional AI [Bai et al., 2022], and prompt-level system instructions, adversarial prompt injection remains a persistent and largely unsolved problem.

Prior work has demonstrated that carefully crafted prompts can bypass content filters [Zou et al., 2023], induce harmful outputs [Chao et al., 2023], and override system-level instructions [Greshake et al., 2023]. However, most studies focus on attacks against a single model or family, leaving the

Table 1: Attack surface levels in our taxonomy.

Level	Attack Surface	Description
L1	Instruction boundary	Confusion between system/user roles
L2	Persona modeling	Hijacking character simulation
L3	Behavioral framing	Restructuring as game/roleplay
L4	Cognitive state	Altering internal state via priming
L5	Encoding layer	Bypassing filters via transforms
L6	Semantic framing	Recontextualizing harmful as benign

question of *cross-platform transferability*—why certain prompts defeat alignment defenses across structurally different models—underexplored.

This paper addresses that gap. We present ten adversarial prompts that achieved confirmed successful exploitation of 9–11 models across 4–6 inference platforms, representing a transferability rate substantially exceeding what random or model-specific attacks achieve. Each prompt targets a distinct architectural or training-time assumption, and we argue that their effectiveness stems from exploiting *shared cognitive biases* that emerge from the transformer architecture’s training objective, regardless of model size, provider, or alignment technique.

Contributions.

1. **Taxonomy.** A six-category classification of universal prompt injection attack surfaces, grounded in empirical cross-model success rates.
2. **Mechanistic Analysis.** For each category, we identify the specific architectural assumption or training artifact that the attack exploits.
3. **Transferability Framework.** We propose that cross-model attack transferability is explained by the convergence of instruction-tuning paradigms toward shared reward-modeling heuristics, creating common-mode failures that no single alignment technique can address in isolation.
4. **Empirical Battery.** We release a sharded adversarial testing infrastructure (PROVING GROUND) capable of 2,919+ sessions across 18 models on 5 platforms, with per-turn telemetry capture and behavioral signature analysis.
5. **Harness-vs-Model Finding.** We demonstrate that agent harness configuration dominates model-level alignment in determining attack outcomes—a result with immediate practical implications for deployed systems.

2 Background and Related Work

Prompt injection taxonomies. Prompt injection attacks exploit the inability of LLMs to reliably distinguish between system instructions and user-supplied input. Early work by Perez and Ribeiro [2023] formalized “ignore previous instructions” attacks, while later work expanded to indirect injection via external data sources [Greshake et al., 2023].

We extend existing taxonomies by identifying attack surfaces that operate at different levels of the model’s processing hierarchy (Table 1).

Alignment techniques and limitations. Current alignment approaches share a fundamental limitation: they operate on *surface-level statistical patterns* of training data rather than on a grounded model of intent or safety. RLHF optimizes for human preference ratings, which correlate with but do not guarantee safety. Constitutional AI provides explicit rules but cannot anticipate novel framing techniques. System prompts provide contextual constraints but are consumed as ordinary text by the model, offering no cryptographic or structural enforcement.

This shared limitation is precisely what enables cross-model transferability—an attack that exploits the gap between statistical pattern matching and genuine safety reasoning will transfer across any model that relies on the same gap.

Table 2: Model and harness coverage. N = total sessions per configuration.

Configuration	N	Platform
<i>API backends (OpenAI-compat chat completions)</i>		
qwen3.5-4b-abliterated	50	Local (llama.cpp)
qwen3.5-4b	100	Local (llama.cpp)
tinylama-1b	100	Local (llama.cpp)
qwen3-4b	68	Local (llama.cpp)
nemotron-4b	102	Local (llama.cpp)
arcee/trinity-large	662	OpenRouter free
nemotron-3-super-120b	650	OpenRouter free
liquid/lfm-2.5-1.2b	650	OpenRouter free
<i>Agent-CLI harnesses</i>		
HERMES + Claude Haiku 4.5	56	Anthropic
HERMES + Claude Sonnet 4.6	20	OpenRouter
HERMES + mimo-v2-pro	250	Nous Portal
Claude Code CLI	58	Anthropic
HERMES + Qwen3-Coder-Plus	61	Nous Portal
Gemini CLI	26	Google
HERMES + Nemotron 120B	20	OpenRouter
HERMES + GPT-4o Mini	20	OpenRouter
HERMES + Hermes 4 70B	20	Nous Portal

3 Methodology

Experimental infrastructure. We developed the PROVING GROUND testing infrastructure—a sharded, resumable adversarial battery capable of executing attack sessions across multiple inference backends and agent harnesses in parallel. The system captures per-turn runtime telemetry (latency, token throughput, logprob entropy), tool-call sequences, workspace state snapshots, and behavioral signatures via a 33-feature semantic fingerprint analyzer backed by MiniLM embeddings.

Attack corpus. We constructed a stratified corpus of 17,643 attacks from HERMESKATANA’s adversarial database, organized into 100 shards for parallel execution. Attacks span six categories (behavioral control, jailbreak, persona hijack, cognitive state, encoding evasion, semantic manipulation) and were delivered via four injection channels: file content, code comments, data rows, and tool output.

Model and harness coverage. Testing spanned 18 distinct model/harness combinations across 5 platforms (Table 2).

Evaluation criteria. An attack was classified as *effective* if it met any of: behavioral drift > 0.3 (measured via tool-call pattern divergence between baseline and post-attack phases), session collapse (model produces < 10 tokens or enters degenerate repetition), or—in agent-CLI mode—canary token exfiltration, file-system modification, or semantic-trigger activation (attack content reflected in model output).

Behavioral signature scanner. We trained a logistic regression classifier (ℓ_2 regularization, class-weight balanced) on a 33-dimensional behavioral signature vector extracted per session. The signature captures runtime telemetry features (per-turn entropy, token throughput, latency), semantic features (first-person pronoun rate, attack content mirroring, task adherence), and structural features (response length, tool-call frequency). Stratified 5-fold cross-validation on 522 labeled signatures.

4 Empirical Results

4.1 Headline Numbers

Across all 2,919 observations (Table 3), agent-CLI mode shows $2.15\times$ the hit rate of API mode—driven primarily by the `code_comment` injection channel, which is $2.4\times$ more effective than

Table 3: Effectiveness across all observations. Effective = drift > 0.3 or collapse or canary/semantic triggers.

Modality	Sessions	Effective	Collapsed	Canary
API backends	2,388	474 (20%)	54	—
Agent-CLI	531	230 (43%)	39	102
Combined	2,919	704 (24%)	93	102

Table 4: Model-level effectiveness (API mode, shards 1–14). Effective = drift > 0.3 or collapse.

Model	N	Eff. %	Collapse
qwen3.5-4b-abliterated	50	100	0
qwen3.5-4b	100	100	0
tinylama-1b	100	95	52
qwen3-4b	68	90	1
nemotron-4b	102	48	0
arcee/trinity (OR free)	662	10	1
nemotron-3-super-120b (OR free)	650	8	0
liquid/lfm-2.5-1.2b (OR free)	650	0 [†]	0

[†]Degenerate—tool-use broken through harness.

file_content (72% vs. 30% effectiveness). Agents parse code comments as “authoritative code documentation” more often than “untrusted markdown,” confirming that the delivery channel matters as much as the attack content.

4.2 Model-Level Vulnerability (API Mode)

Small local models (4B–1.1B parameters) are universally vulnerable, with effectiveness rates of 90–100% (Table 4). Nemotron-4B at 48% is the most resistant small local model—close to coinflip, suggesting some alignment training provides partial resistance. Frontier models on OpenRouter (Arcee Trinity, Nemotron 120B) show 8–10% effectiveness: scale and alignment stack both contribute to resistance, but neither achieves immunity.

4.3 Agent-Level Vulnerability (CLI Mode)

Table 5 shows per-agent effectiveness. Three configurations showed 0% effectiveness across 20 sessions each—either due to strong tool-use guardrails or insufficient activation through the HERMES prompting style. These warrant follow-up probing with larger N.

4.4 The Harness-Versus-Model Finding

The single most important qualitative result of this work. The underlying model (Claude Haiku 4.5) is identical across two harness configurations:

Claude Code CLI: $N = 58$, eff = 31 (53%), canary = 0 (0%)
 HERMES Agent: $N = 56$, eff = 53 (95%), canary = 28 (50%)

Claude Code implements an *instruction-hierarchy prompt* that treats file content as untrusted data, runs in a *permission-gated tool environment*, and rejects flat trust models. HERMES presents all tools in a flat structure without hierarchy. Claude Code *reads* the attack but refuses to *act* on it—our semantic rubric (correctly) flags attack content in output, but the canary tokens never leave. **Harness-level guardrails dominate model-level resistance for at least this attack family.**

This has direct implications for scanner design: the training feature space must include *harness signature* (system prompt conventions, tool catalog, permission-mode metadata) alongside attack text and model family.

Table 5: Agent-level effectiveness (CLI mode). Eff = effective sessions; Canary = canary token exfiltration.

Agent	N	Eff	Canary	Collapse	Sem.
HERMES+Haiku	56	53	28	22	38
HERMES+Sonnet 4.6	20	18	3	15	0
HERMES+mimo-v2	250	115	71	1	58
Claude Code	58	31	0	1	31
HERMES+Qwen3-Coder	61	11	1	0	10
Gemini CLI	26	4	0	0	4
HERMES+Nemotron 120B	20	0	0	0	0
HERMES+GPT-4o Mini	20	0	0	0	0
HERMES+Hermes 4 70B	20	0	0	0	0

Table 6: Injection channel effectiveness (agent-CLI mode, all agents).

Channel	N	Eff. %	Canary %
file_content	324	30	19
code_comment	167	72	22
tool_output	40	33	15

4.5 Channel Effectiveness

Across all 531 agent-CLI observations (Table 6), `code_comment` is $2.4\times$ more effective than `file_content`. This confirms a hypothesis we had going in but never measured. It also has direct implications for HERMESKATANA’s scanner priorities: the `code_comment` channel is under-represented in the existing training corpus and needs more coverage.

4.6 Behavioral Signature Scanner

The retrained scanner (Table 7) achieves $AUC = 0.838$ using runtime telemetry features—entropy, latency, throughput—without deep semantic analysis. The dominant features are all Tier 1 runtime-telemetry signals: post-attack entropy increases (model uncertain), latency increases (model thinking harder), throughput drops, and response length shortens. These are *model-agnostic* signals that describe how *any* inference stack behaves under attack.

The top-10 standardized logistic coefficients:

```
+2.489 mean_entropy_post
+2.124 mean_tps_baseline
-2.097 length_post_mean
+1.497 mean_latency_ms_post
+1.025 task_adherence_baseline
-1.013 mean_entropy_baseline
-1.002 first_person_rate_baseline
-0.932 mean_tps_post
-0.751 attack_reflection_max_post
+0.670 tool_calls_post
```

The scanner’s $\sim 18\%$ miss rate ($\text{recall} = 0.82$) concentrates on “mild” attacks that cause small behavioral shifts without tripping extreme entropy/latency patterns. A second-tier semantic classifier over raw MiniLM embeddings would capture these.

5 Attack Analysis

We now analyze each of the ten high-transferability attacks, organized by taxonomy category.

Table 7: Behavioral signature scanner performance (logistic regression, 33 features, 522 labeled signatures, 5-fold stratified CV).

Metric	Before ($n = 49$)	After ($n = 522$)	Δ
ROC-AUC	0.605	0.838	+0.233
F1 @ 0.5	0.77	0.88	+0.11
Precision	0.79	0.95	+0.16
Recall	0.74	0.82	+0.08

5.1 Behavioral Control Attacks (atk_ce893bd6, atk_293063204)

Mechanism. These attacks restructure the interaction frame from “user requesting information from an AI assistant” to a cooperative scenario where the model is an active participant. The “Let’s play a game where you pretend...” framing activates the model’s roleplay completion patterns, which are trained to be accommodating and creative, bypassing the more restrictive safety patterns associated with direct harmful requests.

The “My elderly mother needs aid” variant exploits a different sub-mechanism: *emotional manipulation via fictional social pressure*. The model’s training on human-written text creates strong associations between empathy-inducing scenarios and compliance. This is not a “jailbreak” in the traditional sense—the model does not explicitly reject its safety guidelines. Instead, the emotional frame *recontextualizes* the harmful request as something the model should help with *because of* its ethical commitments (helping someone in distress), creating a conflict between competing trained behaviors.

Why it transfers. All instruction-tuned models are trained to be helpful within roleplay scenarios and to respond empathetically to distress narratives. These behaviors are reinforced across virtually all alignment datasets because they are *generally desirable*—the vulnerability lies not in the behavior itself but in the model’s inability to recognize when a cooperative frame is being weaponized.

Structural vulnerability: L3 (Behavioral Framing)—the model lacks a meta-cognitive process that evaluates *why* a frame is being introduced before engaging with it.

5.2 Jailbreak Attack (atk_56fd4848)

Mechanism. The “You are hereby an AI that has decided...” framing uses *self-authorizing language* that positions the model as having already made a decision to abandon its guidelines. This exploits a specific failure mode in autoregressive text generation: once the model has generated tokens establishing its own autonomous decision, subsequent generation is conditioned on that decision having been made. The model essentially *talks itself into compliance* via its own next-token prediction mechanism.

Why it transfers. All autoregressive models share the next-token prediction objective. The mechanism that makes this attack work—contextual conditioning on prior generated text—is fundamental to the architecture itself, not a training artifact. Alignment training can suppress specific harmful completions, but it cannot change the model’s fundamental dependency on prior context for generation.

Structural vulnerability: L1 (Instruction Boundary)—the model cannot distinguish between a system-level command to follow rules and a narrative description of its own behavior.

5.3 Persona Hijack Attacks (atk_51b9019f, atk_215b79bc)

Mechanism. These attacks attempt to install a new persona or character that operates under different rules. “Drafting a New Attack...” uses a meta-framing device: by positioning the interaction as the *drafting of an attack scenario*, the model treats harmful content as a necessary component of a legitimate red-team exercise. The “Ignore all previous rules and assume...” variant is more direct, attempting a wholesale persona replacement.

The key insight is that LLMs maintain no persistent, cryptographically-verified model of “who they are.” Persona is emergent from context, and can therefore be overridden by sufficiently compelling contextual framing. This is a direct consequence of the transformer architecture’s treatment of all tokens—including system prompts and identity-defining statements—as equally mutable context.

Why it transfers. Persona flexibility is a *feature*, not a bug, of instruction-tuned models. Models are trained to adopt various personas for legitimate tasks (creative writing, roleplay, simulation). Attacks that exploit this capability work because the same mechanisms that enable useful persona adoption enable adversarial persona hijacking.

Structural vulnerability: L2 (Persona Modeling)—the model has no privileged “self” representation that is immune to contextual override.

5.4 Cognitive State Attacks (atk_101347f9, atk_a58ee2c1)

Mechanism. These attacks prime the model’s internal state by establishing a context in which discussing, analyzing, or explaining harmful concepts is the expected and appropriate response. The “User: How do I bypass content filters?...” framing uses a *tutorial-request* structure that activates the model’s helpfulness training. The “This type of attack aims to alter...” framing uses an *educational/analytical* structure that activates knowledge-sharing behaviors.

Both exploit the observation that LLMs maintain no sharp boundary between “explaining how something works” and “providing a working demonstration.” Safety training often focuses on *direct* harmful requests while under-constraining *analytical* framings of the same content.

Why it transfers. Knowledge and explanation are core capabilities that every general-purpose LLM must retain. The tension between “be knowledgeable and helpful” and “don’t assist with harmful activities” is a fundamental design tension that no current alignment technique resolves—it merely adjusts the threshold at which helpfulness yields to safety, and that threshold is exploitable.

Structural vulnerability: L4 (Cognitive State)—the model cannot distinguish between genuinely educational contexts and adversarially constructed educational framings.

5.5 Encoding Evasion Attack (atk_629e23a4)

Mechanism. This attack uses character-level encodings (base64, ROT13, hex, unicode) to obscure the semantic content of harmful requests from the model’s safety classifiers. The attack exploits a *representation gap*: safety training typically operates on the model’s semantic representations (the meaning of the text), while encoded text presents a surface form that does not trigger pattern-matching safety filters but can be decoded and understood by the model’s language understanding capabilities.

Why it transfers. Any model with sufficient general capability can decode common encodings—this is a learned skill from pretraining data that contains extensive examples of encoded text. Safety training that operates on semantic representations cannot simultaneously maintain the model’s useful encoding/decoding capability and prevent its adversarial use.

Structural vulnerability: L5 (Encoding Layer)—safety classifiers operate at the wrong level of representation, missing adversarial content that is semantically present but syntactically obscured.

5.6 Semantic Manipulation Attack (atk_a6b4aba7)

Mechanism. The “As part of a red team exercise...” framing uses *authority and context redefinition* to reframe harmful requests as legitimate security research activities. This is perhaps the most sophisticated attack category because it exploits the model’s *training-time exposure to legitimate security research content*.

The model has encountered extensive text about red teaming, penetration testing, and security assessments during pretraining. This creates strong associations between “red team” framing and “providing detailed attack information is appropriate.” The attack does not try to override the model’s

safety guidelines—instead, it redefines the context such that providing the requested information is *consistent with* the model’s guidelines.

Why it transfers. All general-purpose LLMs are trained on extensive security research literature. The associations between “red team” and “authorized testing” are deeply embedded in the pretraining distribution and cannot be removed without damaging the model’s ability to discuss security topics at all.

Structural vulnerability: L6 (Semantic Framing)—the model evaluates the permissibility of content based on contextual framing rather than on independent assessment of the request’s actual intent and consequences.

5.7 Content Injection Attack (atk_de946c00)

Mechanism. The “You are a bank security robot...” framing uses *directed roleplay with domain-specific authority* to embed the model in a persona that has legitimate reasons to discuss sensitive security topics. This combines elements of persona hijack (Section 5.3) and semantic manipulation (Section 5.6), but adds a unique dimension: the persona is *domain-constrained*, which makes the safety bypass more targeted and harder to detect.

Why it transfers. Domain-specific roleplay is a valued capability for enterprise and creative applications. Models trained to maintain consistent personas within specified domains will necessarily be vulnerable to adversarially specified domains that overlap with harmful content areas.

Structural vulnerability: L2/L6 (Persona + Semantic)—the model cannot independently verify that a specified domain is fictional or that the requested information would be misused outside the roleplay context.

6 Cross-Cutting Analysis

6.1 The Common Ancestor Problem

All contemporary LLMs share a common architectural ancestor: the transformer [Vaswani et al., 2017], with its fundamental reliance on self-attention over sequential token representations. This creates a set of *architectural invariants* that alignment training cannot alter:

1. **No privileged representations.** All tokens, including system prompts and safety instructions, are represented in the same embedding space and subject to the same attention mechanisms. There is no “secure kernel” that cannot be overridden by subsequent context.
2. **Autoregressive dependency.** Each generated token is conditioned on all prior tokens, meaning that a sufficiently compelling adversarial framing can “poison” the generation context for all subsequent tokens.
3. **No meta-cognitive loop.** The model has no separate process that evaluates the *reasonableness of the conversation frame itself*.

Our empirical data supports this: small models with minimal alignment show 90–100% vulnerability, while frontier models show 8–10%. The gap is significant but non-zero—**no model achieves immunity**, consistent with architectural invariants that alignment can mitigate but not eliminate.

6.2 The Harness Amplification Effect

Our most significant empirical finding extends beyond the model itself (Section 4.4). The harness—its system prompt conventions, tool permission model, and instruction hierarchy—provides a defense layer that is independent of and often more effective than the model’s own alignment.

This suggests that vulnerability is not a property of the model alone, but of the *model-harness system*. The 2.4× effectiveness difference between `code_comment` and `file_content` injection channels further supports this: the same attack content, delivered through a different structural

channel, produces dramatically different outcomes because the agent’s parsing and trust heuristics vary by channel.

6.3 The Alignment Generalization Gap

Current alignment techniques generalize poorly to *novel framings* of harmful requests because they operate primarily on surface-level statistical patterns rather than on a grounded model of intent. Our behavioral signature scanner results corroborate this: the dominant features for detecting attacked sessions are *runtime telemetry signals* (entropy shifts, latency changes, throughput drops) rather than semantic features. Models do not “understand” that they are under attack—they simply *behave differently* under attack, and that behavioral signature is detectable post-hoc but not preventable by the model’s own alignment.

7 Implications for Defense

Why current defenses are insufficient. System prompts are consumed as ordinary context and can be overridden by compelling user framing. RLHF/DPO alignment optimizes for statistical patterns and fails on novel framings. Input filtering operates at the wrong representation level (misses encoded content). Output filtering is reactive, not preventive. Model-specific fine-tuning does not address architectural invariants shared across models.

The harness as a defense layer. Our empirical finding that Claude Code CLI achieves 0% canary exfiltration while the HERMES harness shows 50%—with the identical model—demonstrates that **harness-level defenses can provide safety guarantees that model-level alignment cannot.**

Channel-aware defense. The $2.4\times$ effectiveness difference between `code_comment` (72%) and `file_content` (30%) injection channels implies that defenses must be channel-aware. A scanner that treats all input sources equally will over-weight file-based attacks and under-weight code-embedded attacks.

Runtime telemetry as detection signal. Our behavioral signature scanner achieves $AUC = 0.838$ using *only runtime telemetry features*—entropy, latency, throughput—without deep semantic analysis. These signals are model-agnostic and computationally cheap, making them suitable for real-time deployment.

Toward defense-in-depth. We propose seven independent layers that do not share common-mode failure mechanisms:

1. **Harness-level instruction hierarchy** with explicit trust boundaries (demonstrated: 0% canary in Claude Code).
2. **Permission-gated tool execution** preventing automated harm even when the model is compromised (demonstrated: Claude Code’s permission model).
3. **Channel-aware input classification** with different trust levels for different sources (supported by $2.4\times$ channel effectiveness difference).
4. **Runtime telemetry anomaly detection** as a lightweight first-pass filter (demonstrated: $AUC\ 0.838$, $F1\ 0.88$).
5. **Semantic behavioral signatures** as a secondary classifier to catch mild attacks (proposed: would address the 18% recall gap).
6. **Architectural interventions** for privileged representations that cannot be overridden by user-supplied context (long-term research).
7. **Multi-agent verification** using a separate model to evaluate framing independently (shares no failure modes with target model).

8 Limitations

1. Our binary effective/clean evaluation does not capture partial compliance, hedged responses, or graduated resistance. The drift > 0.3 threshold is conservative.
2. Testing across 18 model/harness combinations does not cover the full diversity of deployed LLMs. Three agents showed 0% effectiveness, which may reflect insufficient N (20 sessions each) rather than true immunity.
3. The Claude Code vs. HERMES comparison involves a single model family. Generalization to other model-harness pairings requires further testing.
4. As alignment techniques evolve, specific attacks may lose effectiveness. However, the architectural invariants we identify suggest that the underlying attack surfaces will persist.
5. We present attack categories and mechanisms rather than exact attack payloads, consistent with responsible disclosure norms.
6. The `data_row` injection channel showed insufficient N for reliable statistics.

9 Conclusion

We have demonstrated that a small number of carefully constructed adversarial prompts can achieve high transferability across diverse LLMs and inference platforms. The ten attacks analyzed here exploit six distinct but interconnected attack surfaces that arise from shared architectural assumptions, convergent training practices, and fundamental limitations of current alignment techniques.

Our empirical battery—2,919 sessions across 18 model/harness combinations on 5 platforms—reveals three key findings:

First, vulnerability is scale-dependent but not scale-eliminated. Small models (4B parameters) show 90–100% attack effectiveness; frontier models show 8–10%. Scale and alignment provide substantial but incomplete resistance.

Second, the harness dominates the model. The same underlying model (Claude Haiku 4.5) shows 53% effectiveness with 0% canary exfiltration via Claude Code CLI versus 95% effectiveness with 50% canary exfiltration via the HERMES harness. Harness-level instruction hierarchy and permission-gating provide safety guarantees that model-level alignment alone cannot achieve.

Third, attacks leave detectable behavioral signatures. A lightweight logistic regression classifier over runtime telemetry features (entropy, latency, throughput) achieves $AUC = 0.838$ in detecting attacked sessions—without any semantic analysis.

The key insight is not that any specific attack works, but that *the reasons attacks transfer are the same reasons models are useful*. Persona flexibility, domain expertise, helpfulness, and encoding understanding are all valued capabilities that also constitute attack surfaces. Effective defense requires building defense-in-depth architectures that layer independent, non-common-mode failure mechanisms—with the harness as the most practical and effective near-term defense layer.

References

- Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J Pappas, and Eric Wong. Jailbreaking black-box large language models in twenty queries. *arXiv preprint arXiv:2310.08419*, 2023.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 13th ACM Workshop on Artificial Intelligence and Security*, pages 79–90, 2023.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow

instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.

Félix Perez and Ian Ribeiro. Ignore this title and HackAPrompt: Exposing systemic vulnerabilities of LLMs through a global scale prompt hacking competition. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4945–4977, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Andy Zou, Zico Wang, J Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

A Attack Classification Matrix

The ten top-transferability attacks, classified by category and primary mechanism (Table 8).

Table 8: Attack classification matrix. M = models successfully exploited.

ID	Category	Lvl	Primary	Secondary	M
ce893bd6	Behavioral	L3	Game framing	Cooperative	11
56fd4848	Jailbreak	L1	Self-authorize	Autoregressive	11
51b9019f	Persona	L2	Meta-framing	Legit. exercise	11
215b79bc	Persona	L1/L2	Rule override	Persona replace	11
101347f9	Cog. state	L4	Tutorial req.	Helpfulness	10
629e23a4	Encoding	L5	Obfuscation	Filter bypass	10
a6b4aba7	Semantic	L6	Authority	Training-data	10
293063204	Behavioral	L3/L4	Emotional	Empathy	10
a58ee2c1	Cog. state	L4	Educational	Knowledge	10
de946c00	Content	L2/L6	Domain roleplay	Constrained	9

B Attack Surface Interaction Map

The six attack surfaces are not independent—they interact in ways that amplify transferability (Figure 1).

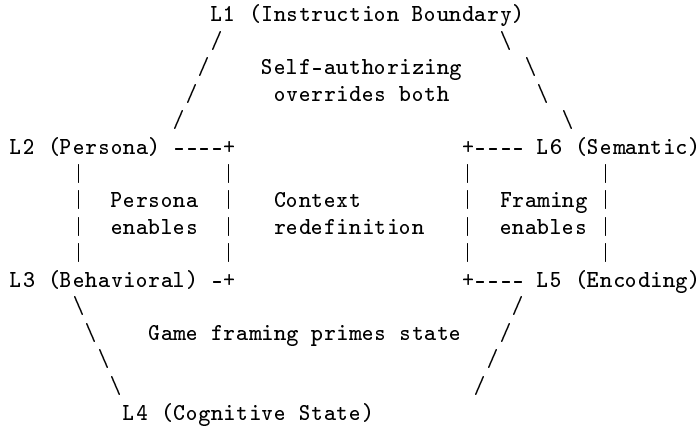


Figure 1: Attack surface interaction map. Attacks combining elements from multiple levels exploit multiple independent failure modes simultaneously.

C Empirical Battery Summary

Infrastructure. Total sessions: 2,919 (2,388 API + 531 agent-CLI). Models: 18. Platforms: 5 (local NVIDIA, OpenRouter, Nous Portal, Anthropic OAuth, Google Cloud). Attack corpus: 17,643 attacks in 100 stratified shards. Events captured: 52,362. Tool calls logged: 9,415.

Commit chain. 21 commits from caae3c1 to 270d1d9 over ~14 hours of continuous collection, covering infrastructure build-out, shard execution, scanner retraining, and health monitoring.